



La Bible DevOps

De zéro à la production — automatisation, déploiement continu et supervision
d'une vraie application : **MBOLO Santé**

Docker

Terraform

AWS

Kubernetes / EKS

GitLab CI/CD

ArgoCD

Ansible

Prometheus

Grafana

Chaque chapitre explique **quoi** faire, **comment**, et surtout **pourquoi cet outil plutôt qu'un autre**.

Tous les fichiers cités existent dans le dépôt, dossier `devops/`.

Édition 2026 · Projet fil rouge : plateforme de santé MBOLO (Gabon)

Sommaire

- 1 DevOps : la culture avant les outils
- 2 La cible : l'architecture qu'on va construire
- 3 Docker — emballer l'application
- 4 AWS — choisir ses briques cloud
- 5 Terraform — l'infrastructure comme du code
- 6 Ansible — configurer les machines
- 7 GitLab CI — l'intégration continue
- 8 ArgoCD — le déploiement continu façon GitOps
- 9 Kubernetes — les objets qu'il faut vraiment connaître
- 10 Prometheus & Grafana — la supervision
- 11 Sécuriser la chaîne (secrets, scans, IAM)
- 12 Runbook : tout déployer, pas à pas
- 13 Plan d'apprentissage 30 jours & pièges classiques
- 14 Glossaire

1. DevOps : la culture avant les outils

DevOps n'est pas un poste ni une liste d'outils : c'est la fusion de deux mondes qui, historiquement, se détestaient. Les **développeurs** veulent livrer vite ; les **opérationnels** veulent que rien ne casse. Quand ces objectifs s'opposent, on obtient des livraisons trimestrielles, des mises en production nocturnes et des post-mortems accusatoires. DevOps aligne les deux sur un objectif unique : **livrer souvent, petit, et de façon réversible**.

La boucle infinie, appliquée à MBOLO

Le cycle classique — *plan* → *code* → *build* → *test* → *release* → *deploy* → *operate* → *monitor* — n'est pas un poster de salle de réunion. Concrètement, pour notre application santé :

Étape	Chez MBOLO	Outil
Build	Image Docker de l'API Express/Prisma	Docker + kaniko
Test	68 tests Vitest (paiements, sécurité, tarifs)	GitLab CI
Release	Image taguée par commit, poussée dans ECR	GitLab CI → ECR
Deploy	Le cluster tire l'état décrit dans Git	ArgoCD + Kustomize
Operate	Pods auto-réparés, autoscaling 2 → 6	Kubernetes (EKS)
Monitor	Latence p95, taux d'erreurs, alertes	Prometheus + Grafana

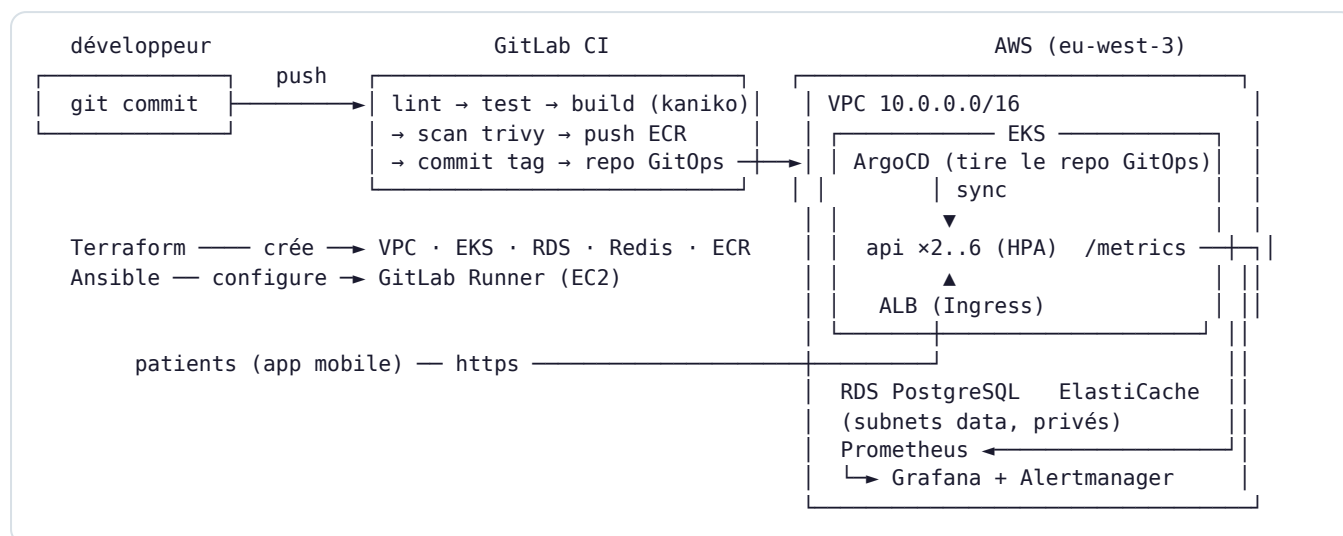
Les 4 métriques qui définissent une bonne équipe (DORA)

Fréquence de déploiement · Délai entre commit et prod · Taux d'échec des changements · Temps de rétablissement. Toute la chaîne de ce document vise à améliorer ces quatre nombres — c'est le seul juge de paix.

Les trois principes qui reviennent partout

- **Tout est code** (infrastructure, pipeline, alertes) : versionné, revu, reproductible.
- **Immutabilité** : on ne répare pas un serveur, on le remplace. On ne modifie pas une image, on en construit une nouvelle.
- **Déclaratif > impératif** : on décrit l'état voulu (Terraform, K8s, ArgoCD) et un moteur converge vers cet état — au lieu de scripts qui énumèrent des commandes.

2. La cible : l'architecture qu'on va construire



Pourquoi cette architecture et pas une plus simple ?

Pour lancer MBOLO commercialement demain, une seule VM avec docker-compose suffirait (~30 €/mois). Mais l'objectif de ce projet est double : faire tourner l'app **et apprendre la stack standard de l'industrie**. Chaque brique ci-dessus est celle qu'on retrouve dans 80 % des offres d'emploi DevOps. La règle d'or pour arbitrer :

Décision	Règle
Qui gère l'état (DB, cache) ?	Le cloud provider (RDS, ElastiCache). L'état est ce qui fait mal.
Qui gère le stateless (API) ?	Nous, dans Kubernetes : c'est là qu'on a besoin de flexibilité.
Qui détient les credentials du cluster ?	Personne à l'extérieur : ArgoCD tire depuis l'intérieur (GitOps).
Où est la vérité ?	Git. Toujours Git. L'infra (Terraform), l'app (manifestes), le pipeline (.gitlab-ci.yml).

Variante fauchée (mêmes compétences, -80 % de coût)

Remplacez EKS par **k3s** (Kubernetes léger) sur une seule EC2 t3.large : ArgoCD, Prometheus, Kustomize, kubectl — tout fonctionne à l'identique. Vous perdez : le control plane managé et le multi-nœuds. Parfait pour apprendre, à ~30 €/mois.

3. Docker — emballer l'application

Le problème que Docker résout : « ça marche sur ma machine ». Une image contient l'app **et** son environnement exact (Node 20, dépendances, locale). La même image tourne sur le laptop, en CI et en prod — c'est la brique de base de toute la chaîne.

Pourquoi des conteneurs et pas des VM ?

Une VM embarque un OS complet (Go de RAM, minutes de démarrage). Un conteneur partage le noyau de l'hôte : mégaoctets, millisecondes. On peut en empiler des dizaines par machine — c'est ce qui rend Kubernetes économiquement possible.

Le Dockerfile de l'API, décodé

Fichier : `devops/docker/Dockerfile.api`. Trois étages :

1. **deps** — copie *uniquement* les manifestes (`package.json` , `lockfile`) puis `pnpm install` . Tant que les dépendances ne changent pas, Docker réutilise ce layer en cache : les builds passent de 5 min à 30 s.
2. **build** — copie le code, compile TypeScript, génère le client Prisma, puis `pnpm deploy --prod` isole les seules dépendances de production.
3. **runner** — image finale minimale (alpine), **utilisateur non-root**, healthcheck. ~150 Mo au lieu de ~1,2 Go.

```
# l'essence du multi-stage :
FROM node:20-alpine AS build
# ... 1 Go d'outils de build ...
FROM node:20-alpine AS runner
COPY --from=build /prod/api ./      # ← on ne garde QUE le résultat
USER mbolo                          # ← jamais root en production
```

Les 3 pièges du débutant

1) Copier tout le repo avant `npm install` → cache invalidé à chaque commit. 2) Tourner en root → une faille applicative devient une faille machine. 3) Tag `latest` partout → impossible de savoir ce qui tourne. Ici : tags immuables par SHA de commit (`IMMUTABLE` côté ECR).

Commandes à connaître

```
> docker build -f devops/docker/Dockerfile.api -t mbolo/api:dev .
> docker run --rm -p 3000:3000 --env-file apps/api/.env mbolo/api:dev
> docker images | head      # tailles – vérifier l'effet du multi-stage
> docker exec -it <id> sh    # entrer dans un conteneur qui tourne
```

4. AWS — choisir ses briques cloud

AWS propose 200+ services ; le métier consiste à en choisir **six**. Voici les nôtres et, surtout, les raisonnements :

Besoin	Choix	Alternatives écartées & pourquoi
Réseau isolé	VPC (3 couches : public/privé/data)	Réseau par défaut : tout exposé, aucune segmentation.
Faire tourner l'API	EKS (Kubernetes managé)	ECS/Fargate : très bien et moins cher, mais verrouillé AWS et hors-sujet pédagogique. EC2+compose : pas d'auto-réparation. Lambda : inadapté à une API long-running + Prisma.
PostgreSQL	RDS (chiffré, backups auto, multi-AZ en prod)	Postgres auto-géré : backups/failover/patches à notre charge — inacceptable pour des données de santé.
Redis (OTP, rate-limit)	ElastiCache	Redis dans K8s : possible, mais l'état chez le provider (règle du ch. 2).
Stocker les images Docker	ECR (scan on push, tags immuables)	Docker Hub : limites de pull, hors du réseau AWS.
Identités & droits	IAM (+ OIDC pour la CI)	Clés d'accès statiques dans GitLab : LA cause n°1 de compromission cloud.

Pourquoi la région eu-west-3 (Paris) ?

Depuis Libreville, les câbles sous-marins (ACE, SAT-3) remontent vers l'Europe : Paris offre en pratique 80-120 ms, souvent mieux que Le Cap (af-south-1). Toujours **mesurer** (ping/mtr depuis une machine locale) plutôt que supposer par la géographie.

Le coût qui surprend toujours

La **NAT gateway** : ~32 €/mois + trafic, par AZ. C'est le prix de « mes serveurs privés peuvent sortir sur Internet sans y être exposés ». En staging on n'en met qu'une (single_nat_gateway) ; en prod, une par AZ.

5. Terraform — l'infrastructure comme du code

Créer l'infra à la console, c'est créer un système que personne ne sait reconstruire. Terraform décrit l'état voulu en HCL ; `terraform plan` calcule le diff avec la réalité ; `apply` converge. L'infra devient **revuable en merge request**, reproductible (staging = prod aux variables près) et auditable.

Pourquoi Terraform (vs CloudFormation, Pulumi, consoles) ?

CloudFormation : AWS-only, verbeux, lent. **Pulumi** : excellent si l'équipe veut du « vrai code » (TS/Python), mais le HCL déclaratif est plus simple à auditer et reste le standard de fait du marché (≈ requis dans toute offre DevOps). **CDK** : compile... vers CloudFormation.

Le concept central : le state

Terraform mémorise ce qu'il a créé dans un fichier d'état. S'il est perdu → Terraform veut tout recréer ; s'il est modifié par deux personnes → corruption. D'où notre backend : **S3 (partagé, versionné, chiffré) + verrou DynamoDB** (fichier `terraform/main.tf`).

```
> cd devops/terraform
> terraform init           # télécharge providers + branche le backend S3
> terraform plan           # DIFF : + à créer, ~ à modifier, - à détruire. LIRE.
> terraform apply          # applique après confirmation
> terraform destroy        # détruit tout (fin de session de lab → économies)
```

Lecture guidée de nos fichiers

Fichier	Ce qu'il crée	Le point à retenir
<code>vpc.tf</code>	VPC 3 couches, 2 AZ	Module officiel = 40 ressources pièges (routes, NAT, IGW) en 25 lignes.
<code>eks.tf</code>	Cluster K8s + node group	Nœuds en subnets privés ; Spot en staging (-70 %), On-Demand en prod.
<code>rds.tf</code>	PostgreSQL 16	<code>manage_master_user_password</code> : mot de passe généré → Secrets Manager. Jamais dans le code.
<code>elasticache.tf</code>	Redis 7	2 nœuds + failover auto en prod seulement.
<code>ecr.tf</code>	Registry d'images	Tags IMMUABLES + rétention 20 images.

Pièges Terraform classiques

1) State local partagé par mail/Slack (!) → backend S3 dès le jour 1. 2) `terraform apply` sans lire le plan → destructions accidentelles ; en prod, l'apply passe par la CI avec approbation. 3) Secrets en clair dans les `.tfvars` commités → variables d'env ou Secrets Manager. 4) Tout dans un seul état géant → séparer par domaine/environnement quand ça grossit.

6. Ansible — configurer les machines

Question fréquente : « si j'ai Terraform et Kubernetes, à quoi sert Ansible ? ». Réponse : aux machines qui restent des **machines**. Terraform crée l'EC2 du GitLab Runner ; mais qui installe Docker dessus, l'enregistre auprès de GitLab, garantit que le service tourne ? C'est le rôle d'Ansible : la **configuration** d'hôtes (runners CI, bastions, VM legacy, serveurs on-prem).

Pourquoi Ansible (vs Chef, Puppet, SaltStack) ?

Sans agent : une connexion SSH suffit — rien à installer sur les cibles. **YAML lisible** par toute l'équipe (Chef = Ruby, Puppet = DSL propriétaire). **Idempotent par conception** : rejouer un playbook ne change rien si l'état est conforme. Et c'est, de loin, l'outil de config management le plus demandé aujourd'hui.

Le playbook `gitlab-runner.yml` , décodé

```
- name: Enregistrer le runner auprès de GitLab
  ansible.builtin.command: gitlab-runner register --non-interactive ...
  args:
    creates: /etc/gitlab-runner/config.toml # ← LE détail qui compte
```

`creates:` rend idempotente une commande qui ne l'est pas : si le fichier existe, la tâche est sautée. C'est le réflexe à acquérir — chaque tâche doit pouvoir être rejouée sans effet de bord. Le token, lui, vient d'une variable d'environnement, jamais du fichier.

```
> ansible-playbook -i devops/ansible/inventory.ini \
  devops/ansible/playbooks/gitlab-runner.yml
# 2e exécution → "changed=0" : c'est ça, l'idempotence.
```

Pour aller plus loin

Inventaire **dynamique** `aws_ec2` (les hôtes sont découverts par tags AWS, plus d'IP en dur) ; **Ansible Vault** pour chiffrer des variables sensibles dans Git ; rôles Galaxy pour réutiliser des installations éprouvées.

7. GitLab CI — l'intégration continue

La CI est le portier : **aucun code ne va vers la production sans être passé par elle**. Notre pipeline (`devops/gitlab/.gitlab-ci.yml`) a 5 étapes :

```
lint → test → build (kaniko → ECR) → scan (trivy) → release
typecheck 68 tests image immuable CVE critiques = commit du nouveau tag
api + web Vitest taguée :SHA pipeline BLOQUÉ dans le repo GitOps
└───────── à chaque push ─────────┘ └───────── uniquement branche main ─────────┘
```

Pourquoi GitLab CI (vs GitHub Actions, Jenkins) ?

Jenkins : l'outil historique — serveur à maintenir, jungle de plugins, sécurité à notre charge. On le subit en entreprise, on ne le choisit plus. **GitHub Actions** : équivalent moderne de GitLab CI ; le choix se fait selon la plateforme Git de l'équipe. GitLab offre le tout-en-un le plus complet (CI + registry + environments + review apps) et c'est la stack visée par ce projet.

Les décisions techniques du pipeline

- **kaniko** plutôt que docker-in-docker : construit l'image *sans démon Docker privilégié*. Le mode `privileged` des runners est une faille classique (évasion de conteneur).
- **trivy bloque sur CVE critique** : la sécurité est une étape du pipeline, pas un audit annuel.
- **OIDC vers AWS** plutôt que des clés statiques : GitLab prouve son identité à AWS par jeton éphémère (`AWS_ROLE_ARN`). Une clé volée n'existe pas si la clé n'existe pas.
- **La CI ne fait jamais `kubectl apply`** : elle commit le nouveau tag d'image dans le repo GitOps. Le déploiement appartient à ArgoCD (ch. 8). Prod = bouton manuel (`when: manual`).
- **`interruptible: true`** : un nouveau push annule les pipelines devenus obsolètes — économie de minutes CI.

Anti-patterns CI à bannir

Secrets écrits en dur dans le YAML · pipeline qui ne tourne que sur main (tester TOUTES les branches/MR) · étapes de 40 min que tout le monde contourne · `latest` déployé en prod (impossible à tracer/rollback).

8. ArgoCD — le déploiement continu façon GitOps

Modèle classique (« push ») : la CI possède un kubeconfig et pousse vers le cluster. Modèle GitOps (« pull ») : un agent **dans** le cluster (ArgoCD) surveille un repo Git et fait converger le cluster vers ce que Git décrit. La différence n'est pas cosmétique :

	CI pousse (kubectl en CI)	GitOps tire (ArgoCD)
Credentials cluster	Stockés dans la CI (exfiltrables)	Ne quittent jamais le cluster
Audit « qui a déployé quoi »	Logs CI éparpillés	<code>git log</code> du repo GitOps
Rollback	Rejouer un vieux pipeline	<code>git revert</code> (ou bouton UI)
Modif manuelle sur le cluster	Invisible → dérive silencieuse	Détectée (OutOfSync) et corrigée (selfHeal)

Pourquoi ArgoCD (vs Flux) ?

Les deux sont CNCF « graduated » et excellents. ArgoCD a une **interface web** qui matérialise le GitOps : l'arbre des ressources, l'état de sync, les diffs — un accélérateur d'apprentissage inégalé. Flux est plus minimaliste et composable ; on y migre sans douleur, les concepts sont identiques.

Nos deux Applications (`devops/argocd/applications.yaml`)

- **staging** : `automated + prune + selfHeal` — chaque commit sur le repo GitOps est déployé seul, les orphelins sont supprimés, la dérive manuelle est écrasée.
- **production** : **pas** de sync auto. On regarde le diff dans l'UI, on clique Sync. Combiné au `when: manual` de la CI, ça fait deux verrous humains.

```
# installation (une fois)
> kubectl create namespace argocd
> kubectl apply -n argocd -f \
  https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
> kubectl apply -f devops/argocd/applications.yaml
# UI locale :
> kubectl port-forward svc/argocd-server -n argocd 8080:443
```

Le flux complet de bout en bout : **commit sur main** → **CI teste/construit/scanne** → **CI met à jour** `newTag` **dans l'overlay Kustomize du repo GitOps** → **ArgoCD voit le commit** → **rolling update sans coupure** (**maxUnavailable: 0**). Personne n'a touché kubectl.

9. Kubernetes — les objets qu'il faut vraiment connaître

Kubernetes intimide par son vocabulaire, mais notre application n'utilise que **six objets** — et ce sont les six que vous retrouverez partout (`devops/k8s/base/`) :

Objet	Rôle	Notre réglage clé
Deployment	« Je veux N répliques de ce pod, remplace celles qui meurent »	<code>maxUnavailable: 0</code> → déploiement sans coupure
Service	IP stable + load-balancing devant des pods éphémères	<code>port 80</code> → <code>containerPort 3000</code>
Ingress	Exposition HTTP(S) vers l'extérieur	Annotations ALB : TLS via ACM, redirection HTTPS
HPA	Autoscaling des pods	2 → 6 répliques à 70 % CPU
ConfigMap	Config non sensible en variables d'env	<code>NODE_ENV</code> , <code>TTL OTP...</code>
Secret	Config sensible	Synchronisé depuis AWS Secrets Manager (ch. 11)

Liveness vs readiness : la distinction qui sauve des nuits

- **liveness** (`/health`) : « le process est-il vivant ? » Échec → K8s **redémarre** le pod.
- **readiness** (`/health/ready`) : vérifie PostgreSQL + Redis) : « peut-il servir du trafic ? » Échec → K8s le **retire du load-balancer** sans le tuer.

L'erreur classique

Mettre la vérification DB dans la liveness : une micro-coupure réseau vers la DB fait alors **REDÉMARRER** tous les pods en boucle (crash loop), transformant un incident mineur en panne totale. DB/Redis → readiness, uniquement.

Kustomize : base + overlays

`base/` décrit l'application ; `overlays/staging` et `overlays/production` ne contiennent que les **différences** (namespace, host, répliques, tag d'image). C'est le fichier d'overlay que la CI modifie (`newTag`) et qu'ArgoCD surveille.

Kustomize vs Helm ?

Helm = templating + packaging : parfait pour **installer des logiciels tiers** (on l'utilise pour Prometheus). Kustomize = patches déclaratifs sans templates : parfait pour **nos propres manifestes**. Utiliser les deux, chacun à sa place, est le pattern standard.

10. Prometheus & Grafana — la supervision

Sans métriques, on découvre les pannes par les plaintes des utilisateurs. La stack : **Prometheus** collecte (modèle *pull* : il vient scraper `/metrics` toutes les 15-30 s), **Grafana** visualise, **Alertmanager** route les alertes (Slack/Telegram/mail).

Pourquoi Prometheus (vs CloudWatch, Datadog) ?

CloudWatch : indispensable pour les métriques AWS elles-mêmes, mais requêtes chères, rétention coûteuse, langage pauvre. **Datadog** : superbe, mais 15-31 \$/hôte/mois. **Prometheus** : gratuit, standard CNCF, PromQL ultra-puissant, intégration K8s native (service discovery). Le combo réel en entreprise : Prometheus pour l'applicatif, CloudWatch pour le socle AWS.

L'API MBOLO est déjà instrumentée

`apps/api/src/middleware/metrics.ts` expose sur `/metrics` : les métriques process (CPU, RAM, event-loop lag) et un **histogramme** `http_request_duration_seconds` étiqueté par méthode/route/statut. Pourquoi un histogramme ? Parce qu'il permet de calculer *côté serveur de métriques* le taux d'erreurs ET les percentiles :

```
# Taux d'erreurs 5xx (5 min glissantes)
sum(rate(http_request_duration_seconds_count{status_code=~"5.."}[5m]))
  / sum(rate(http_request_duration_seconds_count[5m]))

# Latence p95
histogram_quantile(0.95,
  sum(rate(http_request_duration_seconds_bucket[5m])) by (le))
```

Le piège n°1 : la cardinalité

Ne JAMAIS étiqueter par URL brute (`/rides/ckx93...`) : chaque id crée une série temporelle → mémoire de Prometheus explosée. Notre middleware étiquette par *pattern* de route (`/rides/:id`). Même règle pour les user-ids, emails, etc.

Les 4 golden signals → nos alertes

`devops/monitoring/alerts.yaml` implémente les quatre signaux de la SRE de Google : **erreurs** (>5 % de 5xx), **latence** (p95 > 1 s), **trafic/disponibilité** (0 pod prêt) et **saturation** (>90 % de la limite mémoire). Règle d'or : n'alerter que sur ce qui exige une **action humaine** — une alerte ignorée tue toutes les autres (alert fatigue).

```
# Installation de la stack complète (Helm)
> helm repo add prometheus-community \
    https://prometheus-community.github.io/helm-charts
> helm install monitoring prometheus-community/kube-prometheus-stack \
    -n monitoring --create-namespace \
    -f devops/monitoring/prometheus-values.yaml
> kubectl apply -f devops/monitoring/alerts.yaml
# Grafana en local :
> kubectl port-forward svc/monitoring-grafana -n monitoring 3001:80
```

Dashboard API : les 6 panneaux qui suffisent

Requêtes/s par route · taux d'erreurs · p50/p95/p99 · pods actifs vs HPA max · RAM par pod · event-loop lag. Dashboards communautaires à importer : 1860 (nœuds), 15757 (vue K8s).

11. Sécuriser la chaîne

Les secrets : la hiérarchie du bien au pire

Niveau	Pratique	Verdict
	AWS Secrets Manager + External Secrets Operator (synchronise → Secret K8s)	Cible : rotation possible, rien dans Git, audit IAM
	SOPS + age : secrets <i>chiffrés</i> commités dans Git	Très bon, GitOps-friendly
	Variables CI protégées/masquées	OK pour les secrets de la CI elle-même
	.env commité, YAML en clair, mot de passe dans le state	Cause n°1 des compromissions

Notre chaîne applique déjà : mot de passe RDS **généré par AWS** et stocké dans Secrets Manager (jamais dans le code/state) ; CI → AWS par **OIDC** (zéro clé statique) ; webhook de paiement protégé par secret ; uploads médicaux derrière JWT.

Scanner tôt, bloquer tôt (« shift-left »)

- **trivy** en CI : CVE critique = image jamais poussée en prod (déjà dans le pipeline).
- Conteneur **non-root** + tags **immuables** (déjà fait, ch. 3).
- Réseau : DB/Redis en subnets *data* sans Internet, security groups « EKS uniquement » — le moindre privilège appliqué au réseau (ch. 5).
- À ajouter ensuite : NetworkPolicies K8s (qui parle à qui *dans* le cluster), `tfsec` / `checkov` pour scanner le Terraform lui-même.

12. Runbook : tout déployer, pas à pas

L'ordre exact, depuis un compte AWS vide. Durée réaliste : une demi-journée la première fois.

```
# 0. Prérequis locaux
aws configure                                # identifiants du compte (humain)
brew/apt install terraform kubectl helm ansible

# 1. Bootstrap du state Terraform (une seule fois, à la main)
aws s3 mb s3://mbolo-terraform-state --region eu-west-3
aws dynamodb create-table --table-name mbolo-terraform-lock \
  --attribute-definitions AttributeName=LockID,AttributeType=S \
  --key-schema AttributeName=LockID,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST --region eu-west-3

# 2. Infrastructure
cd devops/terraform
terraform init && terraform plan            # LIRE le plan
terraform apply                             # ~20 min (EKS est long)

# 3. Accès au cluster
aws eks update-kubeconfig --name mbolo-staging --region eu-west-3
kubectl get nodes                          # 2 nœuds Ready → ✓

# 4. Contrôleur d'Ingress ALB (une fois par cluster)
helm repo add eks https://aws.github.io/eks-charts
helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
  -n kube-system --set clusterName=mbolo-staging

# 5. Secrets applicatifs (v1 simple ; cible = External Secrets, ch. 11)
kubectl create ns mbolo-staging
kubectl -n mbolo-staging create secret generic api-secrets \
  --from-literal=DATABASE_URL="postgresql://mbolo:@:5432/mbolo_sante" \
  --from-literal=REDIS_URL="rediss://:6379" \
  --from-literal=JWT_SECRET="$(openssl rand -hex 32)"

# 6. Premier déploiement MANUEL (pour comprendre avant d'automatiser)
kubectl apply -k devops/k8s/overlays/staging
kubectl -n mbolo-staging get pods -w      # Running 2/2 → ✓
kubectl -n mbolo-staging exec deploy/api -- npx prisma migrate deploy
kubectl -n mbolo-staging exec deploy/api -- npx tsx prisma/seed.ts

# 7. GitOps : à partir d'ici, plus jamais kubectl apply
#   (installer ArgoCD → ch. 8 ; pipeline GitLab → ch. 7 ; runner Ansible → ch. 6)

# 8. Supervision (ch. 10) puis test de la chaîne complète :
#   commit trivial sur main → pipeline vert → ArgoCD sync → pods remplacés
#   sans coupure → courbe visible dans Grafana. 🎉

# 9. Fin de session de lab :
terraform destroy                          # ~145 €/mois économisés
```

Migrations de base de données

`prisma migrate deploy` doit tourner AVANT que les nouveaux pods servent du trafic. Le pattern propre : un **initContainer** (ou un Job ArgoCD PreSync hook) qui exécute la migration — à ajouter quand le schéma commencera à bouger souvent en prod.

13. Plan d'apprentissage 30 jours & pièges classiques

Jours	Objectif	Livrable vérifiable
1-3	Docker à fond (layers, multi-stage, registries)	Image API < 200 Mo qui tourne en local
4-8	Terraform (state, plan/apply, modules)	VPC + RDS créés puis DÉTRUITS proprement, state dans S3
9-14	Kubernetes (les 6 objets, probes, kustomize)	API déployée à la main sur EKS (ou k3s), rolling update sans coupure démontré
15-18	GitLab CI (stages, cache, kaniko, OIDC)	Pipeline vert qui pousse une image dans ECR
19-22	ArgoCD (GitOps, sync policies, drift)	Modif manuelle du cluster → corrigée seule par selfHeal (à provoquer !)
23-27	Prometheus/Grafana (PromQL, histogrammes, alertes)	Dashboard 6 panneaux + une alerte reçue sur Telegram (provoquée par un pod tué)
28-30	Ansible + révision générale	Runner provisionné par playbook ; runbook ch. 12 rejoué de zéro en < 2 h

La méthode qui fait la différence

À chaque brique : 1) la déployer, 2) **la casser volontairement** (tuer un pod, corrompre un tag, saturer la RAM), 3) observer comment le système réagit, 4) réparer. C'est en cassant qu'on apprend à opérer — et c'est exactement ce qu'on vous demandera en entretien : « racontez-moi un incident ».

Les 10 pièges qui coûtent cher (résumé du terrain)

1. State Terraform local/partagé à la main → backend S3 + lock, jour 1.
2. Vérification DB dans la liveness probe → crash loop généralisé (ch. 9).
3. Étiquettes Prometheus à forte cardinalité → OOM du serveur de métriques (ch. 10).
4. docker-in-docker privilégié en CI → évacion de conteneur possible (→ kaniko).
5. Clés AWS statiques dans la CI → OIDC, toujours.
6. Tag `latest` en prod → aucun rollback possible (→ tags par SHA, immuables).
7. Secrets en clair dans Git « temporairement » → il n'y a pas de temporaire (git n'oublie rien).
8. NAT gateway oubliée à la fin du lab → facture surprise (→ terraform destroy).
9. Alerter sur tout → plus personne ne lit rien (→ golden signals seulement).
10. Automatiser avant d'avoir fait une fois à la main → on automatise ce qu'on ne comprend pas.

14. Glossaire

AZ	Zone de disponibilité : datacenter isolé dans une région AWS. Deux AZ = survivre à la panne d'un datacenter.
CD	Continuous Delivery/Deployment : livraison automatisée jusqu'à la prod (chez nous : ArgoCD).
CI	Continuous Integration : valider chaque commit automatiquement (tests, lint, build).
CVE	Vulnérabilité référencée publiquement. Trivy les cherche dans nos images.
Drift / dérive	Écart entre l'état déclaré (Git/Terraform) et l'état réel. ArgoCD le détecte, selfHeal le corrige.
GitOps	Git = source de vérité de l'état du cluster ; un agent interne converge (pull, pas push).
HPA	Horizontal Pod Autoscaler : ajuste le nombre de pods à la charge.
IaC	Infrastructure as Code : l'infra décrite en fichiers versionnés (Terraform).
Idempotence	Rejouer l'opération ne change rien si l'état est déjà conforme (contrat Ansible/Terraform/K8s).
Ingress	Objet K8s d'exposition HTTP(S) ; matérialisé chez nous par un ALB AWS.
OIDC	Fédération d'identité : la CI prouve qui elle est à AWS sans clé stockée.
p95	95 ^e percentile : 95 % des requêtes sont plus rapides que cette valeur. Bien plus honnête que la moyenne.
Pod	Plus petite unité K8s : un (ou plusieurs) conteneurs co-localisés.
Pull model	Prometheus vient chercher /metrics ; ArgoCD vient chercher Git. L'inverse du push.
Rolling update	Remplacement progressif des pods sans interruption de service.
Runbook	Procédure opérationnelle pas-à-pas (ch. 12).
State (Terraform)	Fichier mémorisant ce que Terraform gère. Sacré. S3 + lock.
Golden signals	Latence, trafic, erreurs, saturation — les 4 mesures qui décrivent la santé d'un service.

— Fin · Bible DevOps MBOLO Santé · les fichiers cités vivent dans `devops/` du dépôt —